

AI Reference — Assignment 02: Web Mario

Student ID: 113062137 | Course: CS2410 Software Studio

1. AI Tool Used

Tool: Claude Code (claude-sonnet-4-6) via Anthropic's Claude Code CLI

Purpose: Full-stack game development assistance — architecture design, code generation, debugging, asset integration, and deployment configuration.

2. How AI Was Used

Area	AI Contribution
Architecture	Designed scene manager, entity system, tilemap structure, physics pipeline
Asset Integration	Wrote plist XML parser for Cocos2d sprite sheets; identified sprite frames via analysis
Physics	Implemented AABB collision detection with minimum-axis resolution
Game Logic	Player states, enemy AI (Goomba/Turtle), question block interactions, power-ups
Level Design	Generated World 1-1 inspired 210-column tilemap with enemies and obstacles
Audio	Web Audio API integration, BGM/SFX management, mute toggle
UI / HUD	Score, coins, timer, lives rendering; all game screens (menu, select, game over, clear)
Deployment	Firebase Hosting setup, firebase.json configuration, multi-commit git history
Debugging	Fixed sprite name suffix bug (.png), array-clear bug on respawn, camera backscroll

3. Example Prompts Given to AI

Initial request: "■■■■AS2_source■■■■ppt■■■■ (Assets are in AS2_source, complete according to PPT requirements)"

Architecture: "Build a pure HTML5 Canvas Mario game with scene manager, AABB physics, sprite sheet loader for Cocos2d plist format"

Bug fix: "Mario is invisible — plist frame keys include .png suffix but config uses names without suffix"

Respawn bug: "Enemies and blocks duplicate after player death — Level._init() pushes onto existing arrays instead of clearing"

Deployment: "Set up Firebase Hosting to deploy the web/ directory as the game's public URL"

4. Files Primarily Written by AI

- web/js/config.js — all constants, animation sequences, sprite sheet definitions
- web/js/loader.js — async plist XML parser and asset loading pipeline
- web/js/spritesheet.js — sprite rendering helpers (draw, drawFit, getSize)
- web/js/physics.js — AABB overlap detection and collision resolution
- web/js/entities.js — Player, Goomba, Turtle, QuestionBlock, BrickBlock, items, effects
- web/js/level.js — World 1-1 tilemap data and level management
- web/js/hud.js — HUD rendering (score, timer, lives, world indicator)
- web/js/scenes.js — all game scenes and scene state machine

- web/js/audio.js — BGM/SFX management with Web Audio API
- web/js/main.js — game loop entry point with asset loading
- web/index.html — HTML5 Canvas entry point
- firebase.json / .firebaserc — Firebase Hosting deployment configuration

5. Summary

This assignment was completed with extensive AI assistance (Claude Code). The AI was used for the full development cycle: designing the game architecture, writing all JavaScript source files, parsing Cocos2d sprite sheet assets, debugging integration issues, designing the level layout, and configuring Firebase deployment. The student provided direction, reviewed the output, and verified gameplay via browser testing.